

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

by

Samuel Nicholas

April 2025

Federated Learning for 3D Medical Image Segmentation: Simulating a Decentralised Network for Healthcare Systems

Project supervisor: Dr Kate Farrahi

Second examiner: Dr Shoaib Jameel

A progress report submitted for the award of MEng Computer Science with Artificial
Intelligence

University of Southampton

Abstract

Faculty of Engineering and Physical Sciences
Electronics and Computer Science

**Federated Learning for 3D Medical Image Segmentation: Simulating a
Decentralised Network for Healthcare Systems**

by Samuel Nicholas

The increasing application of Artificial Intelligence in medical image analysis, particularly for tasks like tumour segmentation, is often hindered by data privacy regulations and the fragmented nature of healthcare data across institutions. Federated Learning (FL) trains robust deep learning models collaboratively without centralising sensitive patient data. This report investigates the implementation and comparison of FL strategies for glioma segmentation using the BraTS 2024 post-treatment MRI dataset. Specifically, we explore the trade-offs between 2D and 3D U-Net architectures within a simulated FL environment using the Flower framework. The standard Federated Averaging (FedAvg) algorithm is compared against Federated Proximal (FedProx), an algorithm designed to mitigate adverse effects of statistical data heterogeneity. Results indicate that while 3D U-Net holds theoretical advantages in leveraging volumetric context, 2D U-Net demonstrated superior practical performance and feasibility due to its substantially lower computational demands, particularly under server instability and time limits. Comparing aggregation strategies for the 2D model, FedAvg achieved slightly higher peak Dice scores than FedProx within the limited rounds conducted. Overall, the study demonstrates the fundamental feasibility of achieving reasonable glioma segmentation accuracy using FL, while highlighting the critical importance of computational efficiency and algorithmic choice for successful deployment in large-scale healthcare settings.

Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Contents

List of Figures	ix
1 Introduction	1
1.1 Background and Motivation	1
1.2 Federated Learning as a Solution	2
1.3 Problem Statement and Research Questions	2
1.4 Objectives	3
1.5 Structure of the Report	4
2 Literature Review	5
2.1 Federated Learning Principles	5
2.1.1 Core Concept and Architecture	5
2.1.2 Data Distribution Scenarios	6
2.1.3 Challenges in Federated Learning	7
2.2 Federated Averaging (FedAvg)	7
2.3 Handling Statistical Heterogeneity: FedProx	8
2.4 Medical Image Segmentation	9
2.4.1 Evolution from CNNs to FCNs and U-Net	10
2.4.2 U-Net Architecture Details	10
2.4.3 3D U-Net for Volumetric Data	11
2.4.4 nnU-Net Framework	11
2.5 The BraTS Dataset for Glioma Segmentation	12
2.5.1 Imaging Modalities	12
2.5.2 Segmentation Labels and Evaluation	13
2.5.3 BraTS 2024: Post-Treatment Challenge	14
2.6 Federated Learning Frameworks: Flower	14
3 Methodology	17
3.1 Dataset Selection and Characterisation	17
3.2 Data Preparation and Preprocessing	18
3.3 Federated Learning System Configuration	19
3.3.1 Framework Implementation: Flower	19
3.3.2 Simulation Architecture: Client-Server Model	20
3.3.3 Data Distribution: Patient-Level Partitioning and Non-IID Data	20
3.3.4 Federated Aggregation Algorithms: FedAvg and FedProx	21
3.4 Segmentation Model Architectures	22
3.4.1 3D U-Net Implementation	22

3.4.2	2D U-Net Implementation	23
3.5	Model Training and Evaluation Procedures	25
3.5.1	Loss Function Formulation	25
3.5.2	Optimisation Algorithm	25
3.5.3	Federated Training Protocol	25
3.5.4	Evaluation Protocol and Metrics	26
3.5.5	Comparative Analysis Framework	26
4	Results	29
4.1	Experimental Setup Summary	29
4.2	Baseline Performance: Single-Client Experiments	30
4.3	Federated Learning Performance: FedAvg Strategy	31
4.4	FedProx vs FedAvg Comparison (2D U-Net)	33
4.5	Qualitative Assessment	35
5	Discussion	37
5.1	Interpreting the 2D versus 3D U-Net Trade-off in Federated Learning . .	37
5.2	Evaluating Aggregation Strategies: FedAvg versus FedProx	38
5.3	Impact of Scalability and the Experimental Environment	39
5.4	Limitations of the Study	40
5.5	Implications and Future Work	41
6	Conclusions	43
Appendix A	Appendix	45
Appendix A.1	Table of Contents for Source Code	45
Appendix A.2	Gantt chart	46
References		47

List of Figures

2.1	A diagram depicting the federated learning cycle	6
2.2	U-Net Architecture where the blue boxes represent multi-channel feature maps	11
2.3	Sample data from the BraTS 2024 dataset. Top left: T1-weighted MRI with contrast. Top right: T1-weighted MRI. Bottom left: FLAIR MRI. Bottom right: T2-weighted MRI	12
2.4	Sample data from the BraTS 2024 dataset with segmentation maps projected onto the data.	13
2.5	Labels of tumour structures: Oedema (yellow), Non-Enhancing Solid Core (red), Enhancing Core (blue), and Cystic/Necrotic Core (green) . .	14
4.1	Loss Curve and Dice Score Curve for 2D U-Net using FedAvg with 10 clients	32
4.2	A comparison between the Validation Dice Scores per federated server round of 2D U-Net and 3D U-Net with 10 clients	33
4.3	Comparison of validation Dice scores per federated server round for the 2D U-Net using FedAvg with different numbers of clients	34
4.4	Comparison of validation Dice scores per federated server round for the 2D U-Net using FedProx with different numbers of clients	34
4.5	Example segmentation result from the 2D U-Net model.	35
4.6	Example segmentation result from the 3D U-Net model.	35
4.7	Example segmentation result from the 2D U-Net model, missing the enhancing/necrotic core	36
4.8	Example segmentation result from the 3D U-Net model, missing the enhancing/necrotic core	36

Chapter 1

Introduction

1.1 Background and Motivation

The integration of Artificial Intelligence (AI), particularly deep learning, into the medical domain has transformed our ability to analyse complex healthcare data. Medical imaging techniques such as Magnetic Resonance Imaging (MRI), Computed Tomography (CT), and X-ray, generate vast amounts of visual data crucial for disease diagnosis, treatment planning, and monitoring [1]. Automating the analysis of these images, especially for tasks such as semantic segmentation, has immense potential to improve diagnostic accuracy, improve workflow efficiency, and support clinical decision making [2]. Segmentation of brain tumours from MRI scans is a particularly critical application area. Gliomas, the most common type of primary malignant brain tumour in adults, exhibit significant heterogeneity in appearance, shape, and location [3]. Accurate segmentation of tumour subregions (e.g., enhancing tumour, tumour core, edema) is vital for grading, surgical planning, radiotherapy targeting, and evaluation of treatment response [4].

Deep learning models, especially architectures like U-Net [5] and its variants [6] [7], have demonstrated high levels of success in medical image segmentation tasks, often achieving performance comparable to, or even exceeding, human experts when trained on large, diverse datasets. However, the practical development of these models faces significant hurdles arising from the inherent nature of medical data. Healthcare data is typically stored within individual hospitals or research institutions, and not usually distributed due to privacy regulations (such as GDPR in Europe or HIPAA in the US), ethical considerations, and institutional data governance policies [8; 9]. This fragmentation prevents the creation of large, centralised datasets required to train highly generalisable deep learning models. Training a model solely on data from a single institution often leads to poor performance when applied to data from

other institutions due to variations in imaging protocols, scanner hardware, and patient populations – a phenomenon known as domain shift.

1.2 Federated Learning as a Solution

Federated Learning (FL) has emerged as a powerful paradigm to overcome the challenges of “data silos” in hospitals and privacy concerns in healthcare [10]. FL allows multiple institutions to collaboratively train a shared model without exchanging their local patient data. In a typical FL setup, a central server coordinates the process. It distributes an initial global model to participating clients (hospitals in this case), each client trains the model locally using its own data, and then they send only the updated model parameters (or gradients) back to the server. The server aggregates these updates (e.g. using Federated Averaging (fedAvg) [10]) to end up with an improved global model, which is then redistributed in the next round of training. This iterative process allows the model to learn from data across multiple clients while ensuring that sensitive patient information remains securely with each individual client [11].

The potential benefits of FL in the medical domain are substantial; allowing the development of more robust, accurate, and generalisable AI models by using larger and more diverse datasets than any single institution could access alone [9; 12]. However, applying FL at scale, particularly within a large, geographically dispersed network, such as the UK’s National Health Service (NHS) with potentially hundreds of participating hospitals, presents its own set of challenges. These include communication bottlenecks, managing straggler clients (clients that are slow to respond), ensuring robustness against client dropouts, and addressing the statistical heterogeneity of data across different sites, which can significantly impact model convergence and performance [13].

1.3 Problem Statement and Research Questions

While FL offers a privacy-preserving solution, its practical application to complex tasks like 3D medical image segmentation in large-scale networks requires careful consideration of architectural choices and algorithmic strategies. Specifically, this project addresses the following key problems and questions:

1. **Architectural Trade-offs (2D vs. 3D U-Net):** Volumetric medical data (like MRI scans) contains rich 3D contextual information. 3D U-Net models [6] are designed to leverage this context directly but, in turn, have substantial

computational costs (memory, processing time). On the other hand, 2D U-Net models [5] operating slice-by-slice, are significantly more lightweight but may sacrifice inter-slice context. In a resource-constrained federated setting, where client hardware capabilities may vary and communication costs are a factor, the optimal architectural choice is unclear. *How does the segmentation performance and computational feasibility of 2D U-Net compare to 3D U-Net when trained within a large-scale federated learning environment?*

2. **Aggregation Strategy Effectiveness (FedAvg vs. FedProx):** The standard FedAvg algorithm [10] simply averages model weights, which can struggle with the statistical heterogeneity inherent in decentralised medical data. FedProx [13] modifies the local client objective to explicitly penalize deviations from the global model, aiming to improve stability and convergence on non-IID data. *Does FedProx offer tangible benefits over standard FedAvg in terms of model accuracy, convergence speed, or stability within the simulated large-scale, non-IID federated network?*
3. **Scalability and Feasibility:** Simulating a network with a large number of clients is crucial for understanding the practical challenges of real-world deployment. *Is it feasible to train effective glioma segmentation models using FL in a simulation involving a large number of clients, and what are the key performance characteristics observed?*

This research aims to answer these questions by implementing and evaluating the different approaches mentioned using the Flower federated learning framework [14] and the BraTS 2024 post-treatment glioma dataset [15].

1.4 Objectives

This project sets forth several key objectives to address the research questions posed. Primarily, it aims to establish a robust FL system utilizing the Flower framework, specifically designed to simulate a large-scale network, thus mimicking the complexity of a national healthcare infrastructure for the task of 3D MRI brain tumour segmentation. The research involves the implementation and adaptation of both 2D U-Net and 3D U-Net segmentation architectures, ensuring their suitability for training within this federated environment using the BraTS 2024 post-treatment glioma dataset. The comparative performance between the standard Federated Averaging (FedAvg) aggregation strategy and the Federated Proximal (FedProx) algorithm, should be analysed with a particular focus on evaluating FedProx's effectiveness in mitigating the challenges posed by data heterogeneity inherent in distributed datasets. Ultimately, this research aims to use these findings to provide valuable insights into

the practical feasibility and effectiveness of implementing various federated learning and deep learning strategies for large-scale, privacy-sensitive medical image segmentation tasks.

1.5 Structure of the Report

This report is structured as follows: Chapter 2 provides a comprehensive review of the relevant background literature on federated learning, algorithms for handling heterogeneity like FedProx, deep learning for medical image segmentation (focusing on U-Net architectures), and the BraTS dataset. Chapter 3 details the methodology employed, including the dataset preprocessing, the federated learning setup using Flower, the specifics of the 2D and 3D U-Net implementations, the loss functions, training procedures, evaluation metrics, and the experimental design for comparing the different approaches. Chapter 4 will present the quantitative and qualitative results obtained from the experiments. Chapter 5 will interpret these results, discuss their implications, acknowledge limitations, and relate them back to the research questions. Finally, Chapter 6 will summarize the key findings and contributions of the research and suggest potential directions for future work. Appendices provide supplementary material, such as the project Gantt chart and the project's code.

Chapter 2

Literature Review

This chapter provides a comprehensive review of the foundational concepts and related work underpinning this research. It delves into the principles of Federated Learning (FL), specific algorithms designed to handle challenges within FL such as FedProx, the evolution of deep learning models for medical image segmentation with a focus on U-Net architectures, the specifics of the BraTS dataset challenge, and the role of FL frameworks like Flower.

2.1 Federated Learning Principles

Federated Learning represents a paradigm shift from traditional centralized machine learning, driven primarily by the increasing importance of data privacy and the distributed nature of data generation in many real-world applications [10][8]. Instead of pooling raw data onto a central server for training, FL enables collaborative model development by keeping data localized on client devices.

2.1.1 Core Concept and Architecture

The fundamental idea involves multiple clients (e.g., hospitals, mobile devices) training a shared machine learning model under the orchestration of a central server. The process typically unfolds iteratively over several communication rounds:

1. **Initialisation and Distribution:** The server initialises a global model (either randomly or from a pre-trained state) and distributes those model parameters to clients.
2. **Local Training and Update Transmission:** Each selected client updates the received model using its local dataset for one or more epochs. This local training

generates updated model parameters specific to the client's data. Clients then transmit their updated model parameters (or related information like gradients, depending on the specific FL algorithm) back to the server. This means that the raw local data is never transmitted.

3. **Aggregation and Iteration:** The server aggregates the received updates from the clients to produce an improved global model. The most common aggregation method is Federated Averaging (FedAvg) [10]. The process then repeats itself from step 2 with the updated global model.

This cycle, depicted conceptually in Figure 2.1 (Sourced from Zhang et al. [11]), continues until the global model converges or a predefined number of rounds is completed. The primary architectural model is the client-server model, although decentralized peer-to-peer FL architectures also exist [16].

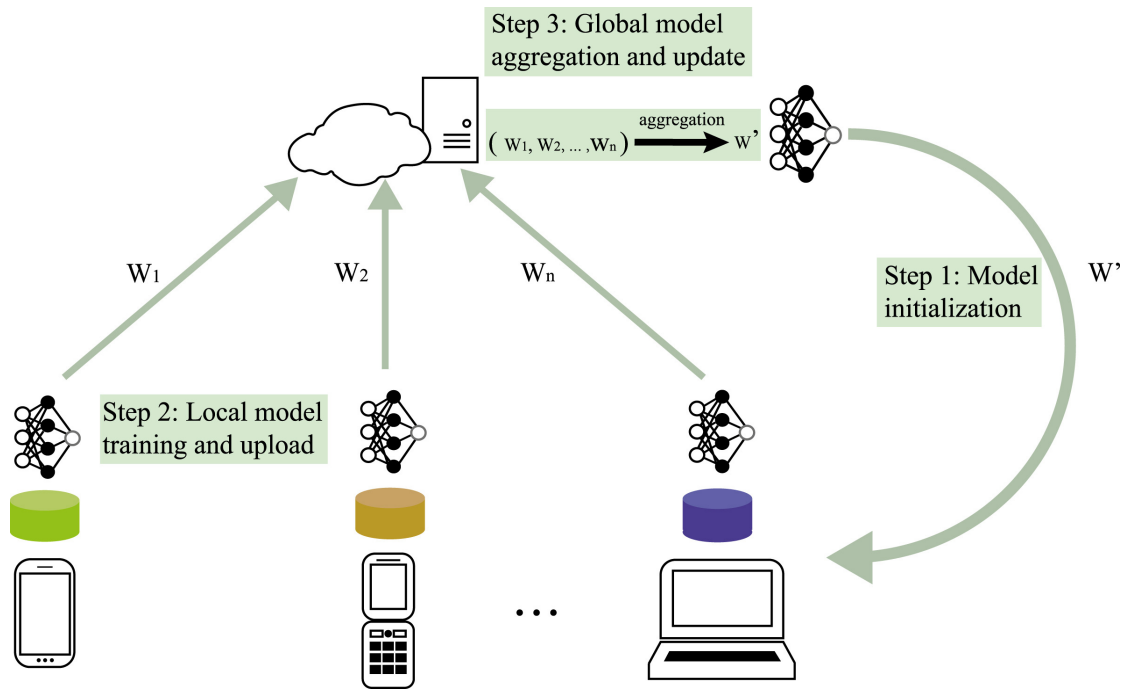


FIGURE 2.1: A diagram depicting the federated learning cycle

2.1.2 Data Distribution Scenarios

Federated learning scenarios are often categorised based on how data is distributed across clients. There are two types of FL scenarios: Horizontal (homogeneous) and Vertical (heterogeneous) [8]. Horizontal learning is where different entities have different samples but share the same feature space; for example, medical institutions holding imaging data of many different patients. Vertical learning, on the other hand, is where different entities have different feature spaces for the same samples; for

example, medical institutions holding many different types of data for the same set of patients [9].

2.1.3 Challenges in Federated Learning

Despite its promise, FL faces several significant challenges, including communication costs, systems heterogeneity (varying client hardware), and privacy/security concerns [16]. One of the most prevalent challenges impacting model performance is statistical heterogeneity. Data distributions across clients are often non-independent and identically distributed (non-IID), arising from differences in patient demographics, disease prevalence, imaging equipment, or data collection practices across institutions. This non-IID nature can cause the local models trained on client data to diverge significantly from one another and from the optimal global model. This divergence, known as ‘client drift’, can impede the convergence of standard aggregation algorithms like FedAvg, leading to slower training, volatility in performance, or convergence to a sub-optimal global model [13]. Addressing statistical heterogeneity is therefore crucial for the success of FL in real-world applications, particularly in healthcare.

2.2 Federated Averaging (FedAvg)

Since each client will have different weights at the end of its training, these need to be aggregated by the server. Federated Averaging (FedAvg) [10] remains the foundational algorithm in Horizontal Federated Learning. FedAvg is built off a variant of stochastic gradient descent (SGD) [17][18] called FederatedSGD (FedSGD) in which a C -fraction of clients are selected each round, and a gradient is computed of the loss over all the data held by the clients. By performing multiple local training epochs on client devices before averaging the resulting model weights, FedAvg reduces the required number of communication rounds compared to synchronizing gradients more frequently. This is achieved by iterating the local update $w^k \leftarrow w^k - \eta \nabla F_k(w^k)$ multiple times before the averaging step [10]. Algorithm 1 (Sourced from McMahan et al. [10]) shows their pseudocode for the FedAvg algorithm, starting from a random initialization. While communication-efficient, this simple averaging mechanism does not explicitly account for client drift caused by non-IID data, potentially limiting its performance in heterogeneous settings.

Algorithm 1 FederatedAveraging

Inputs: The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

```

1: Initialize  $w_0$ 
2: for each round  $t = 1, 2, \dots$  do
3:    $m \leftarrow \max(C \cdot K, 1)$ 
4:    $S_t \leftarrow$  (random set of  $m$  clients)
5:   for each client  $k \in S_t$  in parallel do
6:      $w_t + 1^k \leftarrow \text{ClientUpdate}(k, w_t)$ 
7:   end for
8:    $m_t \leftarrow \sum_{k \in S_t} n_k$ 
9:    $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_t + 1^k$ 
10: end for

```

ClientUpdate(k, w): // Run on client k

```

1:  $B \leftarrow$  (split  $P_k$  into batches of size  $B$ )
2: for each local epoch  $i$  from 1 to  $E$  do
3:   for batch  $b \in B$  do
4:      $w \leftarrow w - \eta \nabla \ell(w; b)$ 
5:   end for
6: end for
7: return  $w$  to server

```

2.3 Handling Statistical Heterogeneity: FedProx

The challenge posed by non-IID data in FedAvg has motivated the development of numerous alternative algorithms. One prominent approach is Federated Proximal (FedProx) [13]. FedProx aims to improve the stability and convergence of federated learning in the presence of statistical heterogeneity while still allowing for varying amounts of local computation by clients.

The core idea of FedProx is to modify the local objective function minimised by each client during their local training phases. Instead of solely minimising the local empirical loss $F_k(w)$ based on the client's data D_k , FedProx introduces a proximal term that penalises large deviations of the client's local model parameters w from the current global model parameters w_t received from the server at the start of the round. The modified local objective function $H_k(w; w_t)$ becomes:

$$\min_w H_k(w; w_t) = F_k(w) + \frac{\mu}{2} \|w - w_t\|^2 \quad (2.1)$$

Where $F_k(w) = \frac{1}{n_k} \sum_{j \in D_k} \ell(w; x_j, y_j)$ is the standard local loss function, $\|\cdot\|^2$ denotes the squared Euclidean norm, and $\mu \geq 0$ is a non-negative hyper-parameter controlling the strength of the proximal term [13].

The proximal term $\frac{\mu}{2}\|w - w_t\|^2$ acts as a regularisation mechanism during local training. By minimising this combined objective H_k , the client is encouraged to find updated weights w_{t+1}^k that not only perform well on its local data, but also stay relatively close to the initial global model w_t . This explicitly limits the extent to which local models can drift away from the global model due to reducing the effect of biased local data on the model.

The hyperparameter μ plays a crucial role in the outcome of model training.

- If $\mu = 0$, the proximal term vanishes, and FedProx essentially reduces to FedAvg.
- If $\mu > 0$, it enforces a degree of similarity between the local models and the global model. A larger value of μ imposes a stronger penalty on deviations, forcing local models to stay closer to w_t . This can improve stability and convergence speed in highly heterogeneous settings by preventing local models from diverging too drastically.
- However, setting μ too large can overly restrict local adaptation, potentially slowing down convergence or preventing the model from reaching the optimal solution that might require significant deviation from the initial w_t to fit the local data.

Finding an appropriate value for μ is often empirical and may depend on the degree of heterogeneity in the specific federated network. After local training, the aggregation step at the server is the same as in FedAvg: the server computes a weighted average of the locally updated weights w_{t+1}^k (which were obtained by minimising H_k) to form the new global model w_{t+1} . By mitigating client drift through the proximal term, FedProx aims to provide more robust and stable convergence compared to FedAvg, particularly when dealing with non-IID data commonly encountered in scenarios like multi-institutional medical imaging. This makes it a relevant algorithm to compare against the baseline FedAvg in this study.

2.4 Medical Image Segmentation

Semantic segmentation, the task of assigning a class label to every pixel or voxel, is fundamental to quantitative medical image analysis. The first segmentation model was proposed by Nobuyuki Otsu in 1979 [19], which entailed a threshold selection method that analyses grey-level histograms to segment simple images. Since then, there have been huge advancements in Image segmentation, and now there are hundreds of different algorithms that are specific to different situations. Now, deep learning (particularly CNN-based approaches) has become the state-of-the-art. There are three main types of models [20]:

1. Structural techniques: Segments each object in an image, focussing on the structure of objects.
2. Stochastic techniques: Labelling local pixels discretely without consideration of any other context.
3. Hybrid techniques: A combination of structural and stochastic techniques. These techniques label each pixel in the image with a class and identify each object instance.

2.4.1 Evolution from CNNs to FCNs and U-Net

Early methods used Convolutional Neural Networks (CNNs) in a sliding-window fashion [21]. Fully Convolutional Networks (FCNs) [22] then enabled end-to-end pixel-wise prediction using an encoder-decoder structure. The U-Net [5] (coined after its U-shaped architecture) significantly advanced this, particularly for medical images, with its symmetric architecture and crucial skip connections merging high-resolution spatial features from the encoder with contextual features from the decoder. This design proved effective even with limited annotated data.

2.4.2 U-Net Architecture Details

The U-Net consists of a contracting path (encoder) and an expansive path (decoder). The encoder uses repeated blocks of 3×3 convolutions (with ReLU/BatchNorm) followed by 2×2 max pooling, doubling feature channels while halving spatial resolution. The decoder uses up-convolutions (transposed convolutions) to increase resolution and halve channels, concatenates features from the corresponding encoder layer via skip connections, and applies double 3×3 convolutions. A final 1×1 convolution maps features to the output classes. Figure 2.2 [5] illustrates this structure.

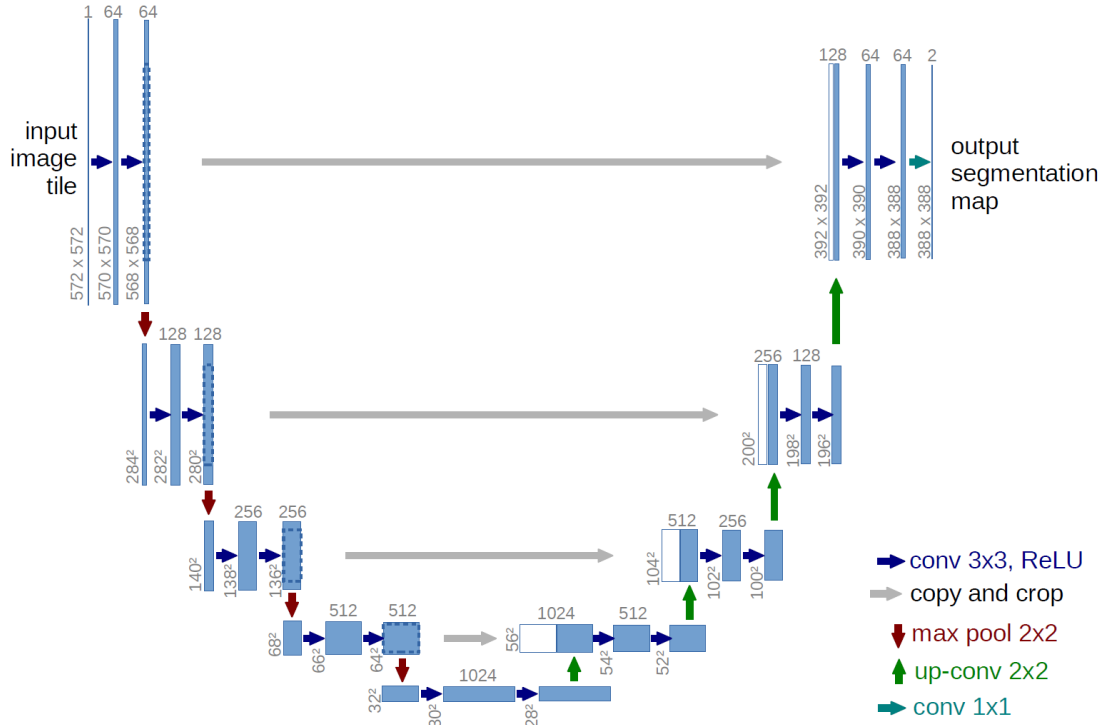


FIGURE 2.2: U-Net Architecture where the blue boxes represent multi-channel feature maps

2.4.3 3D U-Net for Volumetric Data

To leverage inter-slice context in volumetric data like MRI, the 3D U-Net [6] extends the architecture using 3D operations ($3 \times 3 \times 3$ convolutions, $2 \times 2 \times 2$ pooling/transposed convolutions). While capable of learning richer volumetric features, it significantly increases computational and memory demands compared to processing slices with a 2D U-Net.

2.4.4 nnU-Net Framework

The nnU-Net framework [7] automates the configuration of U-Net pipelines (both 2D and 3D) based on dataset characteristics, often achieving benchmark performance. However, its automated nature is less suitable for controlled comparisons of specific architectural choices (2D vs. 3D) as intended in this study.

2.5 The BraTS Dataset for Glioma Segmentation

Gliomas are the most prevalent type of brain tumour in adults and are categorised into four grades, grades I and II being low-grade gliomas (LGG) and grades III and IV being high-grade gliomas (HGG) [3]. HGGs grow rapidly, leading to a high mortality rate. LGGs are usually benign or malignant, meaning they are slow-growing and don't spread to other parts of the body.

Gliomas are usually detected through Magnetic Resonance Imaging (MRI) scans, as they are non-invasive and offer a detailed insight into scanned regions. A sector-leading dataset for segmenting brain tumours is the Brain Tumour Segmentation (BraTS) Challenge dataset, which consists of 3D scans of the brain in NIfTI format (.nii.gz) [4][23][2].

2.5.1 Imaging Modalities

Since gliomas vary in size, shape, and location, as well as being poorly contrasted, it is hard to distinguish healthy tissue from a glioma. This is why the BraTS dataset uses multiple MR modalities, which are T1-weighted MRI (T1), T1-weighted MRI with contrast (T1c), T2-weighted MRI (T2), and Fluid-Attenuated Inversion Recovery (FLAIR) MRI. Figure 2.3 shows an example of a cross-section of a scan taken from the Post-Treatment Glioma Challenge in the BraTS 2024 dataset [12][15].

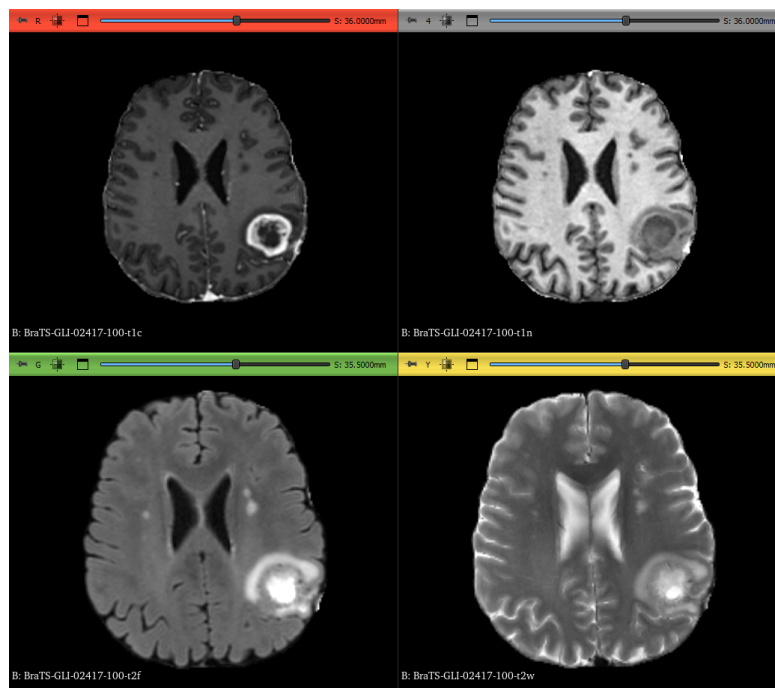


FIGURE 2.3: Sample data from the BraTS 2024 dataset. Top left: T1-weighted MRI with contrast. Top right: T1-weighted MRI. Bottom left: FLAIR MRI. Bottom right: T2-weighted MRI

2.5.2 Segmentation Labels and Evaluation

Expert annotations for the dataset show key subregions: Necrotic/Non-Enhancing Tumour core (NCR/NET), Peritumoral Oedema (ED), and Enhancing Tumour (ET), as shown in Figure 2.4.

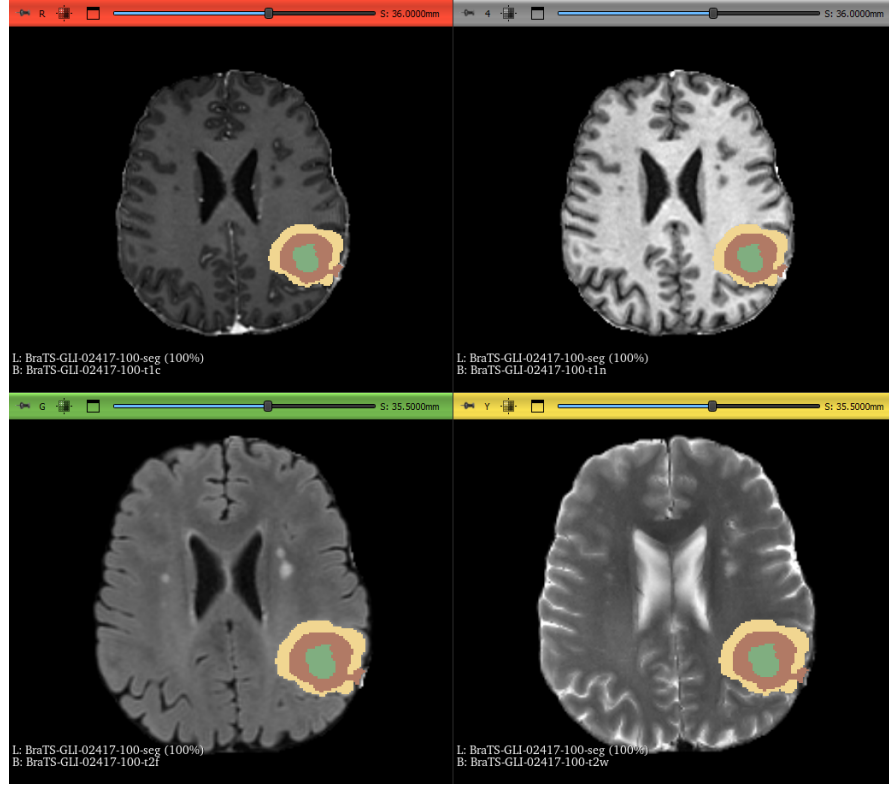


FIGURE 2.4: Sample data from the BraTS 2024 dataset with segmentation maps projected onto the data.

The different subregions are shown by Menze et al. [4] in Figure 2.5. The protocol used to annotate these structures is as follows:

1. The Oedema [Label A] is a build-up of fluid around the glioma, so it can be segmented from FLAIR and T2 images.
2. The Gross Tumour Core that includes enhancing and non-enhancing structures is segmented using hyper-intensities in T1c scans, along with the hypo-intense regions and the inhomogeneous component of the hyper-intense lesion seen in T1 [Label B].
3. The Enhancing Core is segmented by applying a threshold algorithm to the Gross Tumour Core in T1c, excluding the necrotic centre and vessels [Label C].
4. The Cystic/Necrotic Core is defined as the low-intensity structures within the Enhancing Core [Label C].

5. The Non-Enhancing Solid Core is calculated by subtracting the Enhancing Core and Necrotic Core from the Gross Tumour Core [Label D].

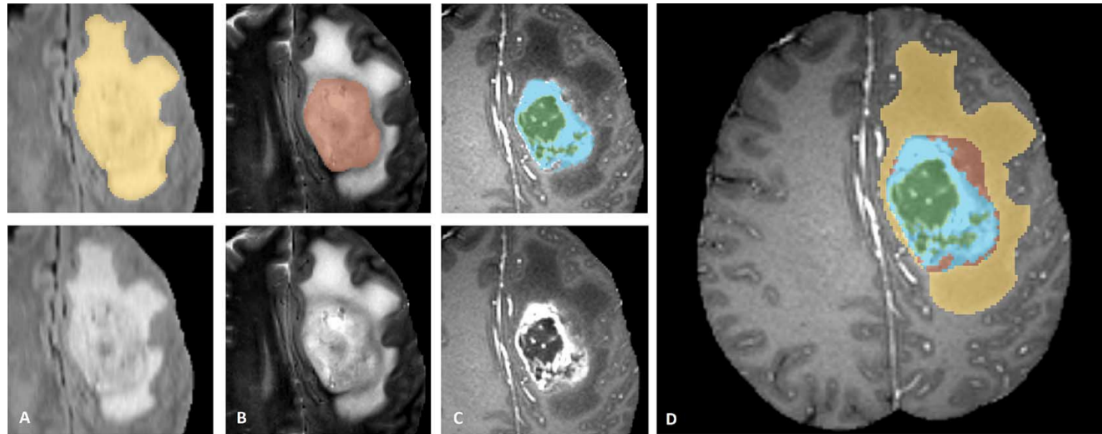


FIGURE 2.5: Labels of tumour structures: Oedema (yellow), Non-Enhancing Solid Core (red), Enhancing Core (blue), and Cystic/Necrotic Core (green)

2.5.3 BraTS 2024: Post-Treatment Challenge

The BraTS 2024 dataset focuses on post-treatment scans [15][12], which introduces significant challenges due to treatment-induced tissue alterations that can mimic or obscure tumour presence, demanding highly robust segmentation models.

2.6 Federated Learning Frameworks: Flower

Federated learning needs specialised software frameworks that handle the underlying complexities of distributed training, communication, client management, and aggregation. Several frameworks exist, including TensorFlow Federated (TFF), PySyft, FATE, and Flower. This project uses Flower (flwr) [14], an open-source framework that's gaining traction due to its design principles emphasising ease of use, flexibility, and scalability.

The server encapsulates client selection, configuration distribution, aggregation, and evaluation triggering within a 'Strategy' object. Flower offers built-in strategies for common algorithms like FedAvg and FedProx, but crucially, it allows users to easily implement their own custom strategies, allowing easy experimentation with novel FL approaches. On the client side, developers implement a Client class that defines how the client interacts with the server and performs local operations. This typically involves methods for receiving parameters, performing local training, evaluating the model, and returning updated parameters or results. This separation between server strategy and client logic promotes modularity and reusability.

Flower also manages the network communication between the server and clients, handling the complexities of serialisation, message passing (often using gRPC), and connection handling. This allows developers to focus on the machine learning aspects rather than low-level networking details. It supports configurations for both simulation environments (where server and clients might run on the same machine) and deployment across geographically distributed systems.

Chapter 3

Methodology

This chapter provides a comprehensive explanation of the methods used in this investigation. The research aims to evaluate the performance and feasibility of applying both 2D and 3D U-Net architectures to the semantic segmentation of gliomas within a simulated federated learning network. The methodology includes dataset selection and preprocessing, the configuration of the federated learning environment using the Flower framework, detailed descriptions of the neural network architectures, the formulation of the loss function, the specifics of client training, and the evaluation protocols established for the analysis of the models.

3.1 Dataset Selection and Characterisation

The dataset of choice for this project is the Brain Tumour Segmentation (BraTS) 2024 Adult Glioma Post Treatment Challenge dataset [4][23][2]. The selection of this specific dataset was influenced by its significant clinical relevance, its status as a premier benchmark within the medical imaging community, and the inherent complexities it presents, which are representative of real-world challenges. Glioma segmentation is a crucial task in neuro-oncology, directly influencing diagnosis, therapeutic strategy formulation, and the monitoring of disease progression [1]. The BraTS challenge provides a standardised platform for evaluating segmentation algorithms, offering high-quality, multi-institutional MRI data accompanied by expert-validated ground truth annotations [4][23]. This standardisation is crucial for ensuring the reproducibility and comparability of results.

A key aspect of the BraTS dataset is its multi-modal nature, providing four different types of MRI scans for each patient case: T1-weighted (T1), post-contrast T1-weighted (T1c), T2-weighted (T2), and T2 Fluid-Attenuated Inversion Recovery (FLAIR). These scans are spatially aligned (co-registered), allowing them to be analysed together.

Each scan type reveals different characteristics of the brain tissue and the tumour. For instance, T1c scans highlight active tumour regions where a contrast agent has accumulated, while FLAIR scans are particularly effective at showing areas of swelling (oedema) around the tumour [23]. The deep learning models developed in this project must learn to effectively fuse the complementary information provided by these four modalities. The specific focus of the BraTS 2024 dataset on post-treatment scans adds further complexity, as treatments can alter tissue appearance, making it harder to distinguish between tumour and treatment effects [15]. This provides a rigorous test for the robustness of the segmentation models under realistic clinical conditions. The data is supplied in the standard NIfTI format (.nii.gz).

3.2 Data Preparation and Preprocessing

Before this data could be fed into the deep learning models, a consistent preprocessing pipeline was necessary to ensure uniformity and suitability for training. This pipeline was implemented using standard Python libraries like NiBabel (for reading NIfTI files) and NumPy (for array manipulation). The first step involved loading the four 3D MRI volumes and the corresponding 3D ground-truth segmentation mask for each patient. Spatial information, such as the affine matrix defining the image orientation and voxel dimensions, was also extracted.

Following loading, intensity normalisation was applied to each 3D MRI volume independently. Because MRI signal intensities can vary significantly between scans due to factors like different scanner settings or patient physiology, normalisation is essential to bring the intensity values to a common scale. This study employed min-max scaling, which rescales the intensity values within each volume to lie between 0 and 1. This standardisation helps stabilise the model training process and makes the model less sensitive to absolute intensity variations.

For the 3D U-Net model, the four normalised 3D volumes (T1, T1c, T2, and FLAIR) were then combined through channel stacking. This process layers the four volumes together, creating a single 4-channel input volume (dimensions: 4 channels x Height x Width x Depth). This structure allows the 3D convolutional filters in the U-Net to process information from all four MRI modalities simultaneously at every point within the volume.

The ground-truth segmentation mask, provided with integer labels indicating different tumour regions (e.g. Label 1 for non-enhancing core, label 2 for oedema, label 4 for enhancing tumour, label 0 for background), was transformed into a one-hot encoded format. This conversion creates a multi-channel mask volume where each channel corresponds to a specific class (e.g., Channel 0: Background, Channel 1: Oedema, Channel 2: Non-Enhancing, Channel 3: Enhancing). In this format, for each

voxel, the channel representing the true class has a value of 1, while all other channels are 0. This one-hot representation is the standard format required for training multi-class segmentation models using common loss functions.

For the experiments involving the 2D U-Net model, an additional preprocessing step was required: 2D slice extraction. The 4-channel 3D input volumes and the corresponding multi-channel 3D masks were sliced along the axial plane (slicing horizontally through the head), generating a dataset composed of individual 2D images (dimensions: 4 channels x Height x Width) and their associated 2D masks (dimensions: 4 classes x Height x Width). Since each 3D scan is comprised of 155 horizontal slices (images are 240 x 240 x 155), this process can generate a very large number of 2D samples, many of which might be redundant or contain little relevant information (e.g., slices with no tumour). Therefore, specific sampling strategies were implemented to create more focused and manageable datasets for each client. These strategies included options to randomly sample a fixed number of slices per 3D volume, or to preferentially select slices known to contain tumour regions for the training set. An overall limit could also be set on the total number of slices assigned to each client. To improve efficiency, especially during the repetitive data loading cycles in federated learning simulations, a slice map caching mechanism was employed. This involves calculating the list of relevant slice identifiers (patient ID and slice index) for each client according to the sampling rules just once, and then saving this list to a file. In subsequent runs, the program can quickly load this cached list, bypassing the time-consuming process of re-examining the 3D volumes.

The final stage of preprocessing involved converting the prepared NumPy arrays (representing either the 3D volumes or the selected 2D slices and their masks) into PyTorch tensors, the standard data structure used by the PyTorch deep learning library [24].

3.3 Federated Learning System Configuration

The configuration of the federated learning system itself is central to simulating the target environment and conducting the comparative experiments effectively.

3.3.1 Framework Implementation: Flower

The Flower (flwr) framework [14] was chosen as the software platform to build and run the federated learning simulation. Flower offers several advantages for this project. It is designed to be flexible and integrate smoothly with popular machine learning libraries like PyTorch, which was used for implementing the U-Net models.

Its structure separates the server's coordination logic from the client's local computation logic, making the system modular and easier to develop. Flower provides implementations of standard algorithms like FedAvg and FedProx, while also allowing for the creation of custom strategies if needed.

3.3.2 Simulation Architecture: Client-Server Model

The simulation adopts a standard client-server architecture, where a single server process acts as the central coordinator. It maintains the current state of the shared global model, selects which clients participate in each round of training or evaluation, distributes the necessary information (like model parameters) to the selected clients, collects the results sent back by the clients, aggregates these results according to the chosen strategy (FedAvg or FedProx), and updates the global model. Multiple client processes are run in parallel, each simulating an independent entity (like a hospital). Each client manages its own local data partition, receives instructions and models from the server, performs local computations (training or evaluation), and sends its results back. The total number of simulated clients is configured to reflect the scale of the network being modelled, but due to hardware limitations and dataset size, the maximum number of clients simulated in this project was 10.

3.3.3 Data Distribution: Patient-Level Partitioning and Non-IID Data

To realistically simulate the way medical data is typically siloed in different institutions, the BraTS training dataset was partitioned among the simulated clients at the patient level. The list of unique patient identifiers was randomly shuffled and divided into distinct subsets according to how many clients were being simulated. Each client was assigned one of these subsets, meaning all MRI scans and segmentation data for a particular patient resided exclusively on one client. This approach ensures data privacy within the simulation (as no raw data is shared) and, crucially, creates a non-independent and identically distributed (non-IID) data distribution across the network. Because different clients hold data from different patient groups, the characteristics of the data (e.g., tumour types, sizes, patient demographics, potential subtle scanner variations) are likely to differ significantly from one client to another. This statistical heterogeneity is a common and challenging aspect of real-world federated learning, and incorporating it into the simulation through this partitioning method allows for a more realistic evaluation of how well different FL algorithms cope with such data diversity.

3.3.4 Federated Aggregation Algorithms: FedAvg and FedProx

The core of the algorithmic investigation lies in comparing two different methods for the server to aggregate the updates received from the clients.

The baseline method is Federated Averaging (FedAvg) [10]. In this approach, after clients perform their local training for a set number of epochs, they send their updated model parameters (w_{t+1}^k) back to the server. The server then computes the new global model (w_{t+1}) by calculating a weighted average of these parameters. The weight assigned to each client's parameters is proportional to the number of data samples it used during its local training relative to the total number of samples across all participating clients in that round. This method is straightforward and widely used but can be sensitive to the 'client drift' problem caused by non-IID data, where local models diverge significantly during local training, and simply averaging them might not yield an optimal global model [13].

To address the limitations of FedAvg, particularly in the context of non-IID data, this study also implements and evaluates Federated Proximal (FedProx) [13]. FedProx modifies the local training process on each client. Instead of only minimising the loss on the local data ($F_k(w)$), clients using FedProx minimise a modified objective function that includes an additional 'proximal' term: $H_k(w; w_t) = F_k(w) + \frac{\mu}{2} \|w - w_t\|^2$. This proximal term measures the squared Euclidean distance between the client's current local parameters (w) and the global parameters (w_t) received from the server at the start of the round. The hyperparameter $\mu \geq 0$ controls the influence of this term. By adding this term, the client's local optimisation is regularised: it is encouraged to find parameters that not only fit the local data well but also stay relatively close to the global model. This mechanism directly counteracts client drift by limiting how far local models can diverge due to heterogeneous local data. When $\mu = 0$, FedProx reduces to FedAvg. When $\mu > 0$, it enforces greater agreement between local and global models, which can lead to more stable convergence and potentially better performance on non-IID data. However, the value of μ needs careful consideration, as too large a value might hinder the model's ability to adapt to local data. After the clients complete their local training using this modified objective, the server aggregates the resulting parameters (w_{t+1}^k) using the same weighted averaging procedure as FedAvg. The key difference lies in the optimisation performed locally on the client. Comparing FedProx (with $\mu > 0$) against FedAvg allows this study to assess the practical benefits of using an algorithm specifically designed to handle statistical heterogeneity in the context of large-scale medical image segmentation.

3.4 Segmentation Model Architectures

This study evaluates two distinct deep learning architectures based on the U-Net design, implemented using the PyTorch library. The choice between these architectures represents a fundamental trade-off between computational cost and the potential benefits of processing data in its native dimensionality. CUDA [25] is utilised to allow training on client GPUs, significantly decreasing the time taken to execute models.

3.4.1 3D U-Net Implementation

The 3D U-Net model, shown in Algorithm 2, is specifically constructed to operate on volumetric data, directly processing the 4-channel 3D input tensors $(4, H, W, D)$ generated from the preprocessed MRI scans. Its architecture follows the established principles of 3D U-Nets [6], featuring a symmetrical encoder-decoder structure connected by skip connections. The encoder path progressively extracts features while reducing spatial resolution using blocks containing double $3 \times 3 \times 3$ convolutions with 3D Batch Normalisation and ReLU activation, followed by $2 \times 2 \times 2$ max pooling. Conversely, the decoder path increases spatial resolution using $2 \times 2 \times 2$ transposed convolutions, concatenates the result with corresponding high-resolution features from the encoder via skip connections, and refines the features using further double $3 \times 3 \times 3$ convolution blocks. The initial number of filters is set to 16, increasing deeper into the network. A final $1 \times 1 \times 1$ convolution maps the high-level features to the 4 output classes (logits) for each voxel. The main advantage of this architecture lies in its inherent ability to learn and utilise the spatial relationships between adjacent slices within the 3D volume. The model for this project is based off the 3D U-Net by Wang et al [26].

Algorithm 2 3D U-Net**Require:** 3D input volume x , classes n_{cls} , base filters f , bilinear flag

```

1: function DOUBLECONV3D( $x, c_{\text{out}}$ )
2:    $x \leftarrow \text{Conv3D}(x; \text{out\_ch} = c_{\text{out}}, k = 3, p = 1)$ 
3:    $x \leftarrow \text{BatchNorm3D}(x)$ 
4:    $x \leftarrow \text{ReLU}(x)$ 
5:   repeat once:
6:      $x \leftarrow \text{Conv3D}(x; \text{out\_ch} = c_{\text{out}}, k = 3, p = 1)$ 
7:      $x \leftarrow \text{BatchNorm3D}(x)$ 
8:      $x \leftarrow \text{ReLU}(x)$ 
9:   return  $x$ 
10: end function
11: function DOWN3D( $x, c_{\text{out}}$ )
12:    $x \leftarrow \text{MaxPool3D}(x; k = 2)$ 
13:   return DOUBLECONV3D( $x, c_{\text{out}}$ )
14: end function
15: function UP3D( $x, \text{skip}, c_{\text{out}}, \text{bilinear}$ )
16:   if bilinear then
17:      $x \leftarrow \text{Upsample}(x; \text{scale} = 2)$ 
18:   else
19:      $x \leftarrow \text{ConvTranspose3D}(x; \text{out\_ch} = \frac{\text{in\_ch}}{2}, k = 2, s = 2)$ 
20:   end if
21:    $x \leftarrow \text{PadToMatch}(x, \text{skip})$   $\triangleright$  center crop or symmetric pad
22:    $x \leftarrow \text{Concat}(\text{skip}, x)$ 
23:   return DOUBLECONV3D( $x, c_{\text{out}}$ )
24: end function
25: function UNET3D( $x$ )
26:    $x_1 \leftarrow \text{DOUBLECONV3D}(x, f)$ 
27:    $x_2 \leftarrow \text{DOWN3D}(x_1, 2f)$ 
28:    $x_3 \leftarrow \text{DOWN3D}(x_2, 4f)$ 
29:    $x_4 \leftarrow \text{DOWN3D}(x_3, 8f)$ 
30:    $x_5 \leftarrow \text{DOWN3D}(x_4, 16f/(1 \text{ or } 2))$ 
31:    $u_1 \leftarrow \text{UP3D}(x_5, x_4, 8f/(1 \text{ or } 2), \text{bilinear})$ 
32:    $u_2 \leftarrow \text{UP3D}(u_1, x_3, 4f/(1 \text{ or } 2), \text{bilinear})$ 
33:    $u_3 \leftarrow \text{UP3D}(u_2, x_2, 2f/(1 \text{ or } 2), \text{bilinear})$ 
34:    $u_4 \leftarrow \text{UP3D}(u_3, x_1, f, \text{bilinear})$ 
35:    $y \leftarrow \text{Conv3D}(u_4; \text{out\_ch} = n_{\text{cls}}, k = 1)$ 
36:   return  $y$ 
37: end function

```

3.4.2 2D U-Net Implementation

In contrast, the 2D U-Net model (shown in Algorithm 3) processes the data as a sequence of independent 2D slices. Each input is a 4-channel 2D tensor ($4, H, W$), representing the four MRI modalities for a single axial slice. The architecture mirrors the original U-Net design [5] but employs 2D operations throughout: 2×2 convolutions, 2D Batch Normalisation, 2×2 Max Pooling, and 2×2 Transposed Convolutions. Skip connections are used similarly to link corresponding encoder and decoder stages. To potentially enhance its capacity to learn complex patterns within each slice, the number of initial filters is set higher (64) compared to the 3D model. The final 1×1 convolution produces the 4-channel output logits for the input slice. The primary rationale for investigating the 2D U-Net is its significantly lower computational complexity. It requires fewer parameters and substantially less

memory than the 3D model, making it potentially much faster to train and more suitable for deployment on clients with limited computational resources, which might be common in a large, diverse network. The central question addressed by comparing these two architectures is the extent to which the performance gained by processing full 3D context outweighs the practical benefits of the computationally lighter 2D approach in a federated setting.

Algorithm 3 2D U-Net

```

1: function DOUBLECONV2D(channels_in, channels_out)
2:   Conv2d(channels_in, channels_out, 3×3, padding=1, bias=False)
3:   BatchNorm2d(channels_out)
4:   ReLU(inplace=True)
5:   Conv2d(channels_out, channels_out, 3×3, padding=1, bias=False)
6:   BatchNorm2d(channels_out)
7:   ReLU(inplace=True)
8: end function
9: function DOWN2D(channels_in, channels_out)
10:  MaxPool2d(2×2)
11:  DoubleConv2D(channels_in, channels_out)
12: end function
13: function UP2D(channels_in, channels_out, bilinear)
14:  if bilinear then
15:    Upsample(scale=2, mode=bilinear, align_corners=True)
16:  else
17:    ConvTranspose2d(channels_in, channels_in/2, 2×2, stride=2)
18:  end if
19:  PadToMatch(skip_feature, upsampled)    ▷ center/symmetric padding
20:  Concatenate(skip_feature, upsampled)
21:  DoubleConv2D(channels_in, channels_out)
22: end function
23: function OUTCONV2D(channels_in, num_classes)
24:  Conv2d(channels_in, num_classes, 1×1)
25: end function
26: function UNET2D(n_channels, n_classes, bilinear, initial_filters)
27:   $f \leftarrow$  initial_filters
28:  factor  $\leftarrow$  2 if bilinear else 1
29:   $x_1 \leftarrow$  DoubleConv2D( $n\_channels$ ,  $f$ )
30:   $x_2 \leftarrow$  Down2D( $f$ ,  $2f$ )
31:   $x_3 \leftarrow$  Down2D( $2f$ ,  $4f$ )
32:   $x_4 \leftarrow$  Down2D( $4f$ ,  $8f$ )
33:   $x_5 \leftarrow$  Down2D( $8f$ ,  $16f/\text{factor}$ )
34:   $x \leftarrow$  Up2D( $16f$ ,  $8f/\text{factor}$ , bilinear) ( $x_5$ ,  $x_4$ )
35:   $x \leftarrow$  Up2D( $8f$ ,  $4f/\text{factor}$ , bilinear) ( $x$ ,  $x_3$ )
36:   $x \leftarrow$  Up2D( $4f$ ,  $2f/\text{factor}$ , bilinear) ( $x$ ,  $x_2$ )
37:   $x \leftarrow$  Up2D( $2f$ ,  $f$ , bilinear) ( $x$ ,  $x_1$ )
38:   $x \leftarrow$  OutConv2D( $f$ ,  $n\_classes$ )
39:  return  $x$ 
40: end function

```

3.5 Model Training and Evaluation Procedures

The process of training the models involves iterative local updates on clients followed by global aggregation, guided by a specific loss function and evaluated using standard metrics.

3.5.1 Loss Function Formulation

To effectively train the segmentation models, a hybrid loss function was adopted, combining the strengths of both Cross-Entropy (CE) loss and Dice loss. This combined loss is formulated as $L_{combined} = \alpha L_{Dice} + (1 - \alpha) L_{CE}$. The CE component focuses on the accuracy of classifying each individual pixel or voxel, comparing the model's predicted class probabilities (derived from the output logits) with the true class label. The Dice loss component directly targets the quality of the segmentation overlap. It is calculated as $1 - \text{Average Dice Similarity Coefficient}$ across the relevant foreground (tumour) classes. The DSC measures the spatial agreement between the predicted segmentation mask and the ground-truth mask for a given class. By combining these two components, typically with equal weighting ($\alpha = 0.5$), the training process benefits from the stable gradient properties of CE, especially in early stages, while also being strongly guided towards producing predictions with high spatial overlap with the target structures, which is the ultimate goal of segmentation.

3.5.2 Optimisation Algorithm

The adjustment of model parameters during the local training phase on each client is handled by an adaptive learning rate optimiser. Specifically, the Adam [27] optimiser was used for the 3D model, and the AdamW [28] optimiser was used for the 2D model, as specified in the client implementation scripts. Both Adam and AdamW are popular choices in deep learning as they adapt the learning rate for each parameter based on estimates of the gradients' moments, often leading to faster convergence than standard Stochastic Gradient Descent (SGD). A base learning rate (e.g., 1×10^{-4}) is specified, which the optimiser then adjusts dynamically throughout the training process.

3.5.3 Federated Training Protocol

The entire federated training process spans a predefined number of communication rounds R . Within each round, the server selects a fraction of the available clients to participate. In this project, this has been set such that the server selects all available

clients. These selected clients first load the current global model parameters from the server. They then perform local training for a specified number of epochs on their respective local data partitions, using the designated optimiser and the combined loss function to update their local model parameters. Following local training, the clients upload their updated parameters back to the server. The server then aggregates these parameters using the chosen strategy (either FedAvg or FedProx) to generate the updated global model for the next round. This cycle of distribution, local training, and aggregation continues for R rounds.

3.5.4 Evaluation Protocol and Metrics

Evaluating the performance of the models throughout the federated training process is essential for understanding convergence and generalisation. The primary metric used for assessing segmentation quality is the Dice Similarity Coefficient (DSC). The DSC provides a measure of overlap between the predicted segmentation mask (P) and the ground-truth mask (G) for a specific anatomical region or tumour sub-component, calculated as $DSC = (2|P \cap G|) / (|P| + |G|)$. This metric is computed for the standard BraTS evaluation regions: Whole Tumour (WT), Tumour Core (TC), and Enhancing Tumour (ET). A higher DSC value, approaching 1, indicates better segmentation accuracy. The 'tumour mean' DSC, representing the average DSC across the foreground tumour classes, is used as the main performance indicator during evaluation logging. The validation loss is also recorded. Evaluation is conducted periodically, typically after each global aggregation round. The server sends the updated global model to its clients, which then run the model on their local validation data (which is a different dataset to training data to detect over-fitting). The calculated validation loss and DSC scores are reported back to the server, which aggregates them to provide an estimate of the global model's performance on unseen data across the network. This allows for monitoring the learning progress and comparing the effectiveness of the different experimental setups.

3.5.5 Comparative Analysis Framework

The core of the methodology lies in the systematic comparison of performance across different configurations. By analysing the trajectories of the validation DSC scores and loss values over the federated training rounds, this study aims to draw conclusions regarding the trade-offs between the 2D U-Net and 3D U-Net architectures (when trained with FedAvg) in terms of segmentation accuracy versus computational demands in the FL context. Furthermore, the comparison between FedAvg and FedProx (using 2D U-Net) will illuminate the impact of the aggregation strategy on

convergence stability and final performance, particularly considering the non-IID nature of the distributed data.

Chapter 4

Results

This chapter presents the empirical results obtained from the federated learning experiments conducted according to the methodology outlined in Chapter 3. The performance of the 2D and 3D U-Net architectures, along with the comparison between Federated Averaging (FedAvg) and the Federated Proximal (FedProx) algorithm, is evaluated based on segmentation accuracy and convergence behaviour within the simulated large-scale environment using the BraTS 2024 dataset.

4.1 Experimental Setup Summary

In this chapter, Federated 'Rounds' refers to the server-side act of aggregating a global model. 'Epochs' refer to local client training cycles. When 'Peak' Dice score is mentioned, it is referring to the local peak in the range of the training rounds. Table 4.1 shows the default parameters for the experiments, along with the specific values used for the 2D U-Net and 3D U-Net runs. Since 3D-UNet uses full 3D scans as input, and 2D-UNet uses individual slices, the number of slices processed by 2D-UNet has been increased accordingly. If a parameter is given as 'X', that parameter does not exist in that model. There are 155 2D 'Slices' and each 3D 'Volume'.

It is important to note the context in which these experiments were conducted. The models were ran on a remote High Powered Computing unit, which has options to run on GPU or CPU. To use GPU, your request to run code had to wait in line for an extremely long time (8 hours) to be able to execute. This is an issue because you can only delegate one client to one GPU job, so clients need to wait for enough free GPUs, which causes them to fall out of sync. Due to these complications, all training was performed using CPU resources on a remote server. This significantly impacted execution times, particularly for the 3D U-Net model, making the model train in almost a week instead of a couple of days. Furthermore, the remote server exhibited

Parameter Name	Default	2D U-Net	3D U-Net
Max Slices	20	300	X
Max Samples	40	X	60
Slices per Volume	20	20	X
Rounds (for server)	10	15	30
Require Tumour in Training Slice	False	True	X
Number of Workers	4	4	4
Number of Clients	2	1/2/10	1/2/10
Epochs (for clients)	3	5	5
Batch Size	2	5	2
Learning Rate	$1e^{-4}$	$1e^{-4}$	$1e^{-4}$
Save Frequency	1	1	1
Save Visualisations	False	True	True
proximal μ (FedProx only)	0.01	0.01	X

TABLE 4.1: Default parameters, along with 2D-UNet and 3D-UNet parameters

instability during longer runs, notably affecting multi-client runs where some clients would randomly stop running, preventing their trained weights from being aggregated into the global model. The server imposed a maximum task duration of 60 hours; consequently, the longer-running 3D U-Net experiments exceeded this limit. To mitigate data loss from premature termination, model checkpoints were logged by clients during training, allowing for potential continuation. The results presented here reflect the completed rounds that utilise these checkpoints after instability or disconnection occurred. These factors, especially server instability and the reliance on CPU processing, should be considered when interpreting the performance metrics. Additionally, results for the 2-client 3D U-Net (FedAvg) configuration are not available due to time constraints and storage issues on the remote server.

4.2 Baseline Performance: Single-Client Experiments

To establish a baseline, experiments were conducted simulating a scenario with only a single client. This allows for a direct comparison between the 2D and 3D U-Net architectures and the FedAvg versus FedProx algorithms without the immediate complexities of federated communication and aggregation across multiple clients, although still subject to the CPU training environment.

The 2D U-Net trained with FedAvg achieved a final validation Dice score of approximately 0.645 after 15 rounds, peaking in round 12 (Dice of 0.658). The corresponding final validation loss was 0.401. When the 2D U-Net was trained with FedProx, it achieved a slightly higher final validation Dice score of 0.656, peaking in round 14 (Dice of 0.679), with a final validation loss of 0.421. This suggests minimal difference between FedAvg and FedProx in the single-client setting for the 2D model over 15 rounds. It should also be noted that FedProx had a validation Dice score of

0.605 in the first round, potentially indicating the validation images are some of the “easier” slices to segment (mainly unaffected brain tissue).

The 3D U-Net, trained with FedAvg in the single-client setting, achieved a final validation Dice score of 0.670 after 30 rounds, peaking in round 28 (Dice of 0.674). Its final validation loss was 0.330. Comparing the single-client runs, the 3D U-Net marginally outperformed the 2D U-Net (FedAvg) in terms of validation Dice score and achieved a lower validation loss by the end of training. All model results that are not federated are displayed in table 4.2.

TABLE 4.2: Summary of Non-federated models

Configuration	Peak Validation Dice Score	Final Validation Dice Score	Final Loss
2D U-Net	0.658	0.645	0.401
3D U-Net	0.674	0.669	0.330

Crucially, a stark difference in computational efficiency was observed, aligning with expectations. As noted previously, the 2D U-Net processed data substantially faster, averaging approximately 3.1 seconds per image (slice), whereas the 3D U-Net required an average of 63.3 seconds per image (volume). While this is not a direct comparison as one model is using volumetric data and the other is using a 2D slice, these times directly correlate to how long the model takes to run. The vast difference in processing time, exacerbated by the use of CPU, directly impacted the feasibility of running extended experiments due to time constraints, particularly for the 3D model.

4.3 Federated Learning Performance: FedAvg Strategy

Moving to a federated environment with multiple clients introduces communication overhead and the challenge of aggregating potentially diverse local models. Performance under the standard FedAvg aggregation strategy was evaluated for both 2D and 3D models with varying numbers of clients.

For 2D U-Net using FedAvg, performance was originally going to be compared across 1, 2, and 10 simulated clients. But since there are no results for 3D U-Net across 2 clients, only results from 1 and 10 client simulations will be compared. The 10-client run yielded a final validation Dice score of 0.668. The corresponding final validation loss was 0.361. This indicates a modest increase in segmentation accuracy and a slight increase in validation loss as the number of participating clients increased. The learning curve, illustrated conceptually in Figure 4.1, shows this trend.

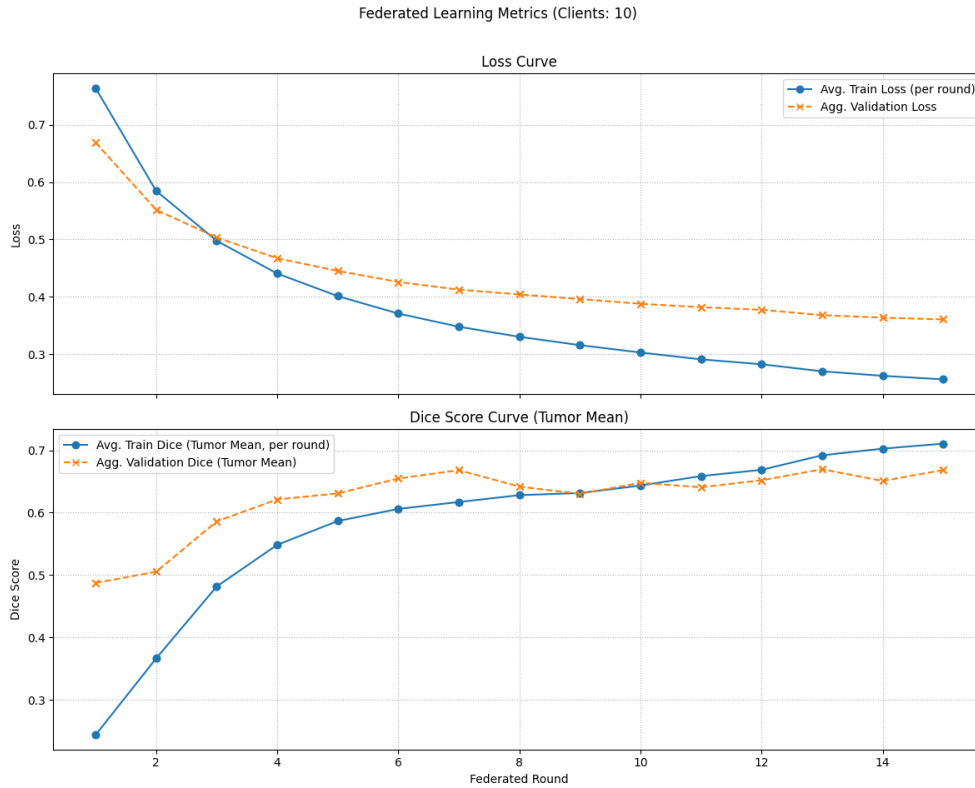


FIGURE 4.1: Loss Curve and Dice Score Curve for 2D U-Net using FedAvg with 10 clients

The 3D U-Net using FedAvg was evaluated with 10 clients. As mentioned, this run was affected by server instability, with an average of 9 clients participating per round and clients disconnected and reconnected throughout training. The final and peak validation Dice score achieved after 30 rounds was 0.527, with a final validation loss of 0.352. This represents a noticeable drop compared to the single-client 3D baseline (0.674 Dice score). While the increased client count and non-IID data distribution likely contributed to this performance decrease, the server instability and potential early termination due to time limits are significant confounding factors for this specific run.

Comparing the 10-client runs directly (shown in Figure 4.2, the 2D U-Net (FedAvg) achieved a considerably higher final validation Dice score (0.668) than the 10-client 3D U-Net (FedAvg) (0.527). This difference is more pronounced than in the single-client baseline comparison, potentially exaggerated by the computational demands and instability issues faced by the 3D model in the federated, CPU-constrained environment.

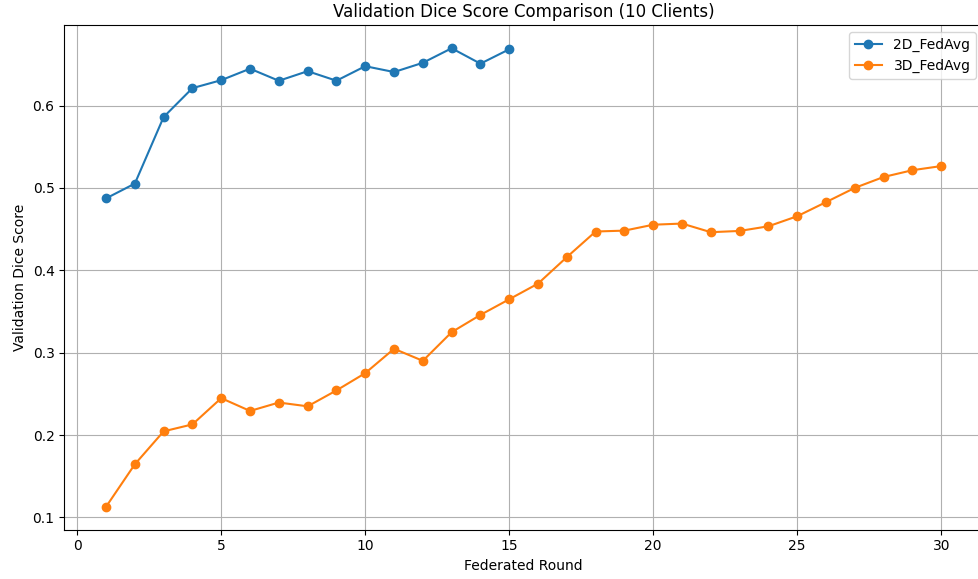


FIGURE 4.2: A comparison between the Validation Dice Scores per federated server round of 2D U-Net and 3D U-Net with 10 clients

4.4 FedProx vs FedAvg Comparison (2D U-Net)

The FedProx algorithm was compared against FedAvg for the 2D U-Net architecture to assess its impact on handling the non-IID data distribution inherent in the client partitioning.

In the single-client scenario, as noted earlier, FedProx achieved a final validation Dice score of 0.679, compared to FedAvg's 0.658. The difference is minimal over 15 rounds, suggesting that in the absence of aggregation challenges and significant client drift (as there is only one client), the proximal term offered only a slight advantage.

With 2 clients, the FedProx run resulted in a final validation Dice score of 0.662, while the FedAvg run achieved 0.640. Again, in this limited 2-client setup over 15 rounds, FedProx marginally outperformed FedAvg based on the final Dice score. For the 10-client scenario, FedAvg barely outperforms FedProx with a final validation Dice score of 0.668 compared to 0.664. A closer look at the learning curves (shown in Figures 4.3 and 4.4) gives us insight into the stability and convergence of both architectures. FedProx has a smoother convergence compared to FedAvg, which shows that it is more resistant to large deviations from the global model. Table 4.3 contains a summary of all models' Peak Validation Dice Scores.

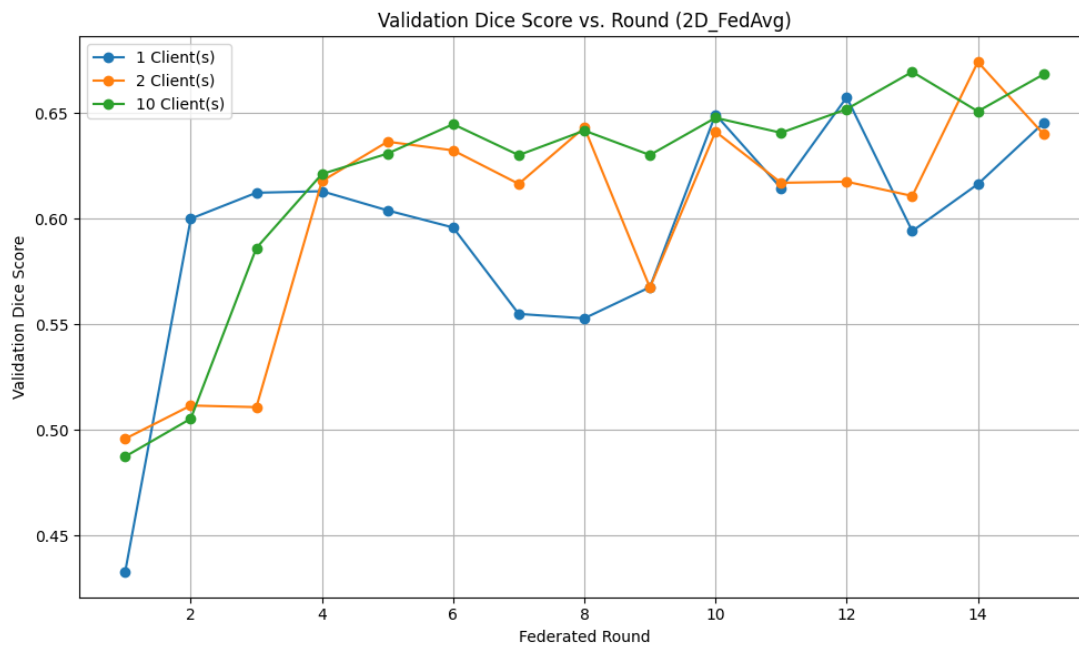


FIGURE 4.3: Comparison of validation Dice scores per federated server round for the 2D U-Net using FedAvg with different numbers of clients

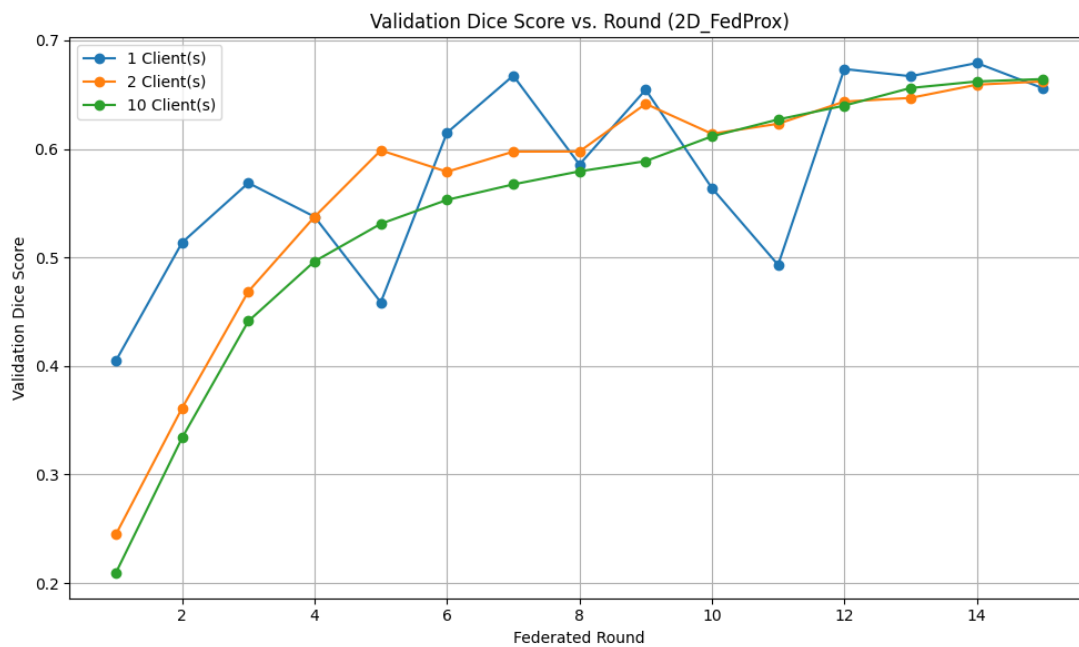


FIGURE 4.4: Comparison of validation Dice scores per federated server round for the 2D U-Net using FedProx with different numbers of clients

TABLE 4.3: Summary of Peak Validation Dice Scores

Configuration	Clients	Algorithm	Peak Validation Dice Score
2D U-Net	1	FedAvg	0.658
2D U-Net	1	FedProx	0.679
3D U-Net	1	FedAvg	0.674
2D U-Net	2	FedAvg	0.674
2D U-Net	2	FedProx	0.662
2D U-Net	10	FedAvg	0.670
2D U-Net	10	FedProx	0.664
3D U-Net	10	FedAvg	0.527

4.5 Qualitative Assessment

Beyond the quantitative metrics presented above, an inspection of the segmentation outputs produced by the models provides qualitative insight into their performance. Figures 4.5 and 4.6 show examples of these visual results.

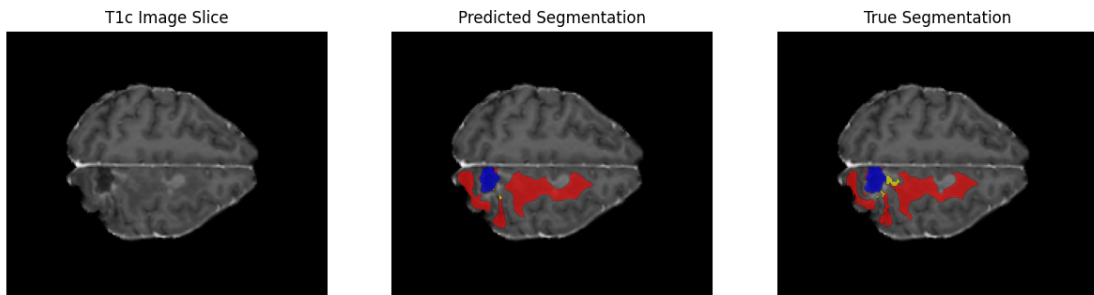


FIGURE 4.5: Example segmentation result from the 2D U-Net model.

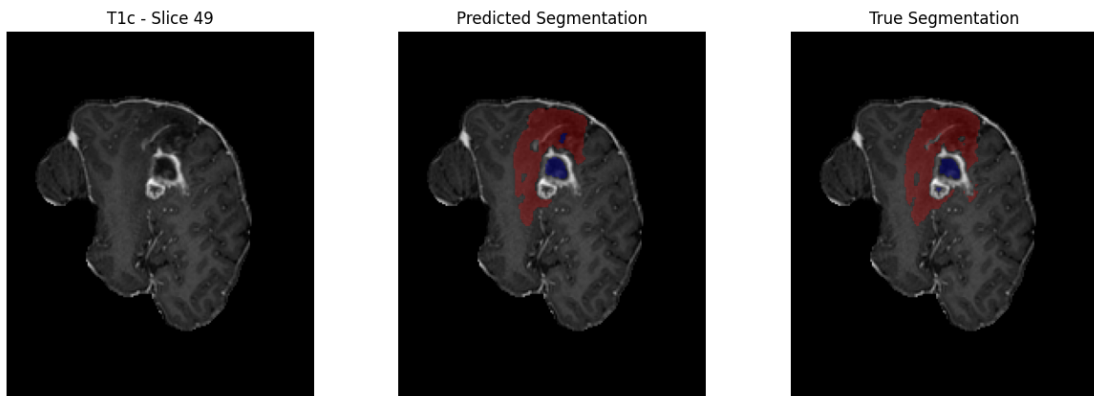


FIGURE 4.6: Example segmentation result from the 3D U-Net model.

Both models are able to segment the oedema (red) and non-enhancing solid core (blue) reliably, but seem to struggle when trying to segment the enhancing/necrotic core (yellow) as shown in Figures 4.7 and 4.8

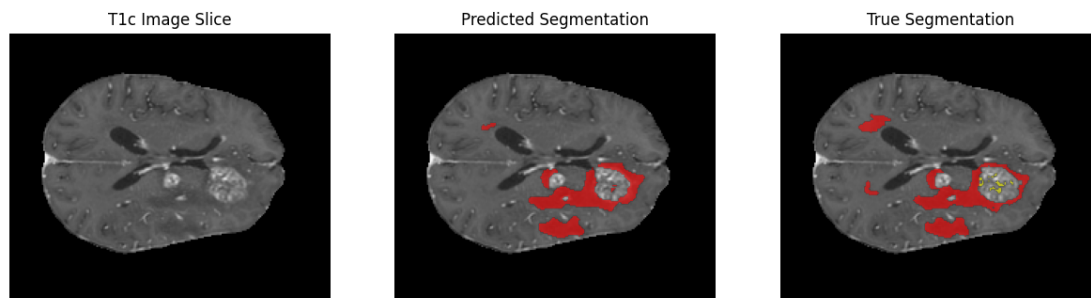


FIGURE 4.7: Example segmentation result from the 2D U-Net model, missing the enhancing/necrotic core

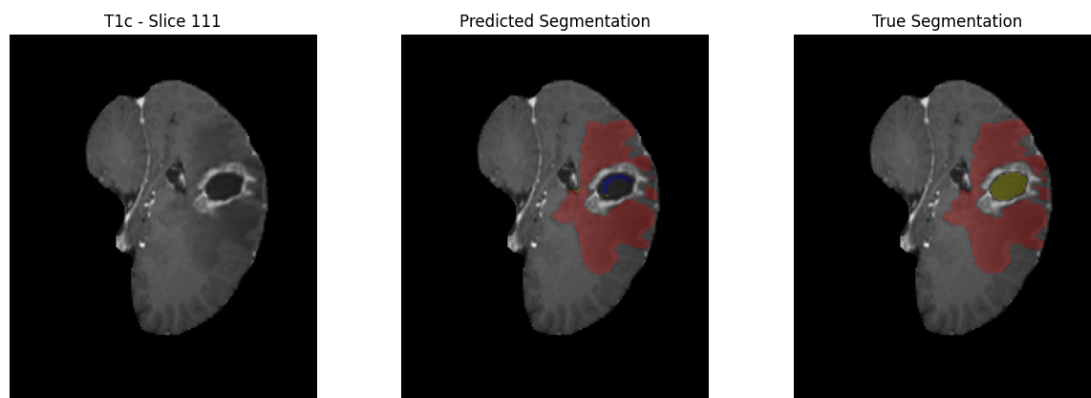


FIGURE 4.8: Example segmentation result from the 3D U-Net model, missing the enhancing/necrotic core

Chapter 5

Discussion

This chapter delves into the interpretation and significance of the experimental results presented in Chapter 4. The findings include the comparative performance of 2D and 3D U-Net architectures, the evaluation of Federated Averaging (FedAvg) against Federated Proximal (FedProx), and the overall viability of employing federated learning for glioma segmentation at scale are analysed. Furthermore, this discussion acknowledges the constraints encountered during the experimental phase and considers the implications of the outcomes for future research and potential real-world applications in decentralised healthcare networks.

5.1 Interpreting the 2D versus 3D U-Net Trade-off in Federated Learning

The comparison between the 2D and 3D U-Net models within the federated learning simulation gave insight into the balance between architectural complexity, computational demands, and segmentation performance under specific experimental conditions. In the single-client baseline scenario, which isolates model performance from federated aggregation challenges, 3D U-Net (FedAvg) achieved a slightly higher peak validation Dice score (0.674) compared to 2D U-Net variant (FedAvg peak 0.658) after completing its longer 30-round run. This suggests that, given sufficient training time and resources, the 3D architecture's ability to process volumetric context directly can offer a marginal advantage in segmentation accuracy, even when trained solely on CPU resources.

However, this advantage diminished significantly, and the practicalities shifted dramatically, when moving to the multi-client federated setting. In the 10-client experiments, the 2D U-Net trained with FedAvg maintained a respectable peak performance (Dice score 0.670), slightly improving on its single-client FedAvg

baseline. Conversely, the 10-client 3D U-Net (FedAvg) experienced a substantial performance degradation, achieving a peak Dice score of only 0.527. This marked decline cannot be solely attributed to the increased data heterogeneity across 10 clients; the experimental context provided in Chapter 4 points towards significant contributing factors.

The primary factor is the difference in computational efficiency. The reported average processing times (approximately 3.1 seconds per slice for 2D vs. 63.3 seconds per volume/patch for 3D on CPU) illustrate the massive computational burden imposed by the 3D architecture. In a time-constrained environment (60-hour server limit) and without GPU acceleration, the 3D model simply could not undergo as effective or extensive training within each federated round or across the total duration compared to the 2D model. This likely resulted in poorer local model quality being aggregated at the server.

Furthermore, the longer execution time per round made the 3D U-Net runs more susceptible to the documented server instability and client disconnections. The 10-client 3D run specifically suffered from reduced average client participation (9 instead of 10) and interruptions, further disrupting the learning process and the quality of the aggregated global model. The checkpointing mechanism mitigated complete data loss but could not fully compensate for the instability's impact on consistent training progress. Due to time constraints, specialising the models to fit this task specifically was not a possibility, making the models difficult to compare to similar models outside of the project.

Consequently, despite the theoretical appeal of processing 3D context, the 3D U-Net proved less practical and ultimately less effective than the 2D U-Net within this simulated, resource-constrained, and somewhat unstable federated environment. The 2D U-Net, because of its computational efficiency and robustness to environmental challenges, ended up as the more viable and higher-performing architecture under these specific conditions. This finding shows the importance of considering not just the sophistication of model architecture, but also its computational footprint and resilience when designing FL systems for potentially heterogeneous and resource-limited real-world scenarios. For large-scale networks where client capabilities vary and stability cannot always be guaranteed, a computationally lighter, robust 2D approach might offer a more reliable pathway to achieving good segmentation performance.

5.2 Evaluating Aggregation Strategies: FedAvg versus FedProx

The comparison between the standard FedAvg algorithm and the heterogeneity-aware FedProx algorithm was conducted using the more practical 2D U-Net architecture.

The aim was to determine if FedProx’s proximal term, designed to limit client drift on non-IID data, provided tangible benefits in this setting.

In the single-client baseline, FedProx achieved a slightly higher peak Dice score (0.679) than FedAvg (0.658) over 15 rounds. While seemingly counterintuitive, as the proximal term has no client drift to counteract, this small difference might suggest a minor regularisation benefit from the proximal term itself in this specific run, or it could simply be within the range of normal experimental variation.

In the more critical 10-client scenario, where data heterogeneity effects are expected to be more pronounced, FedAvg achieved a peak Dice score of 0.670 (round 13), while FedProx reached a peak of 0.664 (round 11). Based purely on the peak Dice score achieved within the 15 rounds, FedAvg performed slightly better. However, FedProx’s primary proposed benefit is enhanced stability and robustness against divergence, rather than necessarily achieving a higher peak score in all situations, especially over a limited number of rounds. Examining the learning curves presented (Figures 4.3 and 4.4) offers some visual context. While both algorithms show fluctuations, it is clear to see that FedProx offered superior stability. It is possible that with more rounds, or with more pronounced data heterogeneity (which might occur with even larger client numbers), the stabilising effect of FedProx might translate into better long-term performance. Based on the available 15-round results, however, a clear advantage for FedProx over FedAvg was not demonstrated in terms of peak segmentation accuracy for the 10-client 2D U-Net configuration. The choice of the μ parameter (set to 0.01 for these experiments) also influences FedProx’s behaviour and might require further tuning for optimal results in this specific application.

5.3 Impact of Scalability and the Experimental Environment

The experiments simulating 1, 2, and 10 clients provide insights into the impact of network scale on performance, although these insights are heavily influenced by the challenging experimental environment. For the 2D U-Net trained with FedAvg, increasing the client count from 1 to 10 led to a slight increase in the peak validation Dice score achieved within 15 rounds (from 0.658 to 0.670). This contrasts with the expectation that performance might degrade due to increased data fragmentation and non-IID effects. This outcome could potentially be explained by the model benefiting from exposure to a more diverse range of data samples present across the 10 clients compared to the single client’s partition, outweighing the negative effects of simple averaging over this number of rounds. Figure 4.3 confirms that the 10-client run reached a slightly higher peak than the 1-client run.

The scalability assessment for the 3D U-Net is severely hampered by the performance drop observed in the 10-client run (peak Dice 0.527) compared to the single-client run

(peak Dice 0.674). As discussed, this significant degradation is likely a combined effect of non-IID data challenges, the computational demands of the 3D model on CPU, and server instability. It strongly suggests that scaling the 3D U-Net architecture using standard FedAvg within such a constrained and unstable environment is highly problematic.

The overall performance levels achieved, particularly the peak Dice scores around 0.66-0.68 for the 2D U-Net configurations, indicate that federated learning is capable of training reasonably effective glioma segmentation models on the challenging BraTS 2024 post-treatment dataset, even under suboptimal conditions (CPU training, potential instability). While these scores might be lower than state-of-the-art results achieved with centralised training on powerful GPUs (Dice scores of about 0.80 [26]), they demonstrate the fundamental feasibility of the federated approach for this task while preserving data privacy. The gap compared to centralised models highlights the overheads and challenges still present in FL, such as communication costs (implicitly limiting rounds), aggregation effects, and sensitivity to environmental factors.

5.4 Limitations of the Study

It is essential to acknowledge the limitations that temper the conclusions drawn from this research. Firstly, the entire study was conducted within a simulated federated environment. While efforts were made to partition data realistically based on patient IDs to induce non-IID distributions, the simulation cannot fully replicate the complexities of a real-world deployment across numerous hospitals. Factors such as variable network latency and bandwidth, unpredictable client availability and dropout patterns, diverse client hardware capabilities beyond just CPU/GPU differences, and software version inconsistencies were not explicitly modelled.

Secondly, the reliance on CPU resources for all training significantly constrained the experiments. This disproportionately affected the 3D U-Net, limiting the number of rounds that could be completed within the server's 60-hour time limit and likely preventing the model from reaching its full potential performance. The observed computational times clearly indicate that GPU acceleration would be essential for practical application of the 3D model.

Thirdly, the server instability encountered, particularly during the 10-client runs, introduced significant noise and potential bias into the results for that configuration. Client disconnections and the reliance on checkpoints make it difficult to isolate the impact of the federated algorithm itself from environmental failures.

Fourthly, the unavailability of results for the 2-client 3D U-Net leaves gaps in the comparative analysis. The assessment of 3D U-Net’s effectiveness at a small federated scale could not be completed.

Fifthly, the findings are inherently tied to the specific BraTS 2024 dataset and the task of post-treatment glioma segmentation. The conclusions regarding the 2D vs. 3D trade-off or the relative performance of FedAvg vs. FedProx could differ for other medical imaging tasks, modalities, or datasets with different characteristics.

Finally, the use of federated learning algorithms was limited to FedAvg and FedProx. Numerous other advanced algorithms designed to handle heterogeneity or improve communication efficiency exist but were not evaluated. Similarly, while FL provides baseline privacy through data localisation, advanced privacy-enhancing techniques like differential privacy [29] or secure aggregation [30] were not implemented. The hyperparameter tuning, including the choice of the FedProx μ value (0.01), was also limited.

5.5 Implications and Future Work

Despite these limitations, the study offers valuable practical insights for applying federated learning to medical image segmentation. The clear demonstration of the 2D U-Net’s superior feasibility and robustness compared to the 3D U-Net under resource constraints and instability highlights the critical need to balance model complexity with the practicalities of distributed deployment. For large-scale initiatives like a federated NHS network, prioritising computationally efficient and resilient architectures may be essential for successful implementation, at least initially.

The comparison between FedAvg and FedProx highlights the importance of considering algorithms designed for data heterogeneity. While FedAvg performed adequately in the smaller-scale 2D runs, the known theoretical advantages of FedProx in stabilising convergence warrant its further investigation, particularly at larger scales.

Future research should prioritise overcoming the current study’s limitations. Conducting experiments with adequate GPU resources is essential for a fair evaluation of the 3D U-Net’s potential in an FL setting. More research into FedProx is needed, including systematic tuning of the μ parameter and comparison across more clients and rounds. Exploring other advanced FL algorithms specifically designed for non-IID data, such as Scaffold [31], could yield further performance improvements. Incorporating formal privacy guarantees through techniques such as differential privacy [29] is a critical step toward clinical translation. Validating the findings on diverse medical imaging datasets and tasks would strengthen the generalisability of

the conclusions. Ultimately, transitioning from simulation to real-world studies involving collaboration between institutions is necessary to understand the true challenges and potential of federated learning in transforming AI in healthcare.

Chapter 6

Conclusions

This research investigated the application of federated learning (FL) to the complex task of segmenting post-treatment gliomas from multi-modal MRI scans, using the BraTS 2024 dataset within a simulated decentralised network environment managed by the Flower framework. The primary goals were to assess the feasibility of this approach and to compare the effectiveness of different deep learning architectures (2D U-Net vs. 3D U-Net) and federated aggregation algorithms (Federated Averaging - FedAvg vs. Federated Proximal - FedProx).

The experimental findings clearly demonstrated the significant practical challenges associated with deploying computationally intensive models like the 3D U-Net in a resource-constrained (CPU-only) and potentially unstable federated simulation. Despite its theoretical capacity to utilise full volumetric context, the 3D U-Net's performance was severely hampered by its computational demands and sensitivity to server disruptions, resulting in lower segmentation accuracy compared to its 2D counterpart in the multi-client setting. The 2D U-Net, leveraging its much greater computational efficiency, proved more robust and achieved superior segmentation results (peak validation Dice score of 0.670 in the 10-client FedAvg run) under the experimental conditions. This highlights a critical insight for practical FL deployment: architectural choices must carefully balance theoretical sophistication against computational feasibility and resilience within the target distributed environment.

The comparison between FedAvg and FedProx using 2D U-Net yielded less definitive results within the scope of the completed experiments (up to 15 rounds). While FedProx is designed to improve stability on non-IID data, FedAvg achieved slightly higher peak Dice scores in both the 2-client and 10-client scenarios tested. However, given the limited number of rounds, a conclusive assessment of FedProx's potential benefits, particularly regarding convergence stability over longer training horizons or under more extreme heterogeneity, requires further investigation.

Overall, this study successfully demonstrated the fundamental feasibility of training effective glioma segmentation models using federated learning on a challenging, clinically relevant dataset. The peak Dice scores achieved by the 2D U-Net configurations (around 0.66-0.68) indicate that FL can enable collaborative model development while preserving data privacy, although performance likely remains below that achievable with ideal centralised training using GPU resources. The research highlights the practical importance of model efficiency and algorithmic choice in designing scalable FL systems for healthcare.

Future work should prioritise addressing the limitations encountered. Repeating key experiments with adequate GPU acceleration is crucial for a fair evaluation of the 3D U-Net. A comprehensive investigation of FedProx, including systematic tuning of the μ parameter and completion of larger-scale runs over more rounds, is necessary to fully understand its stability characteristics. Exploration of other advanced FL algorithms robust to heterogeneity and the integration of formal privacy-enhancing techniques are also important next steps. Finally, validating these approaches on diverse datasets and transitioning toward real-world studies are essential for bridging the gap between simulation and clinical application.

Appendix A

Appendix

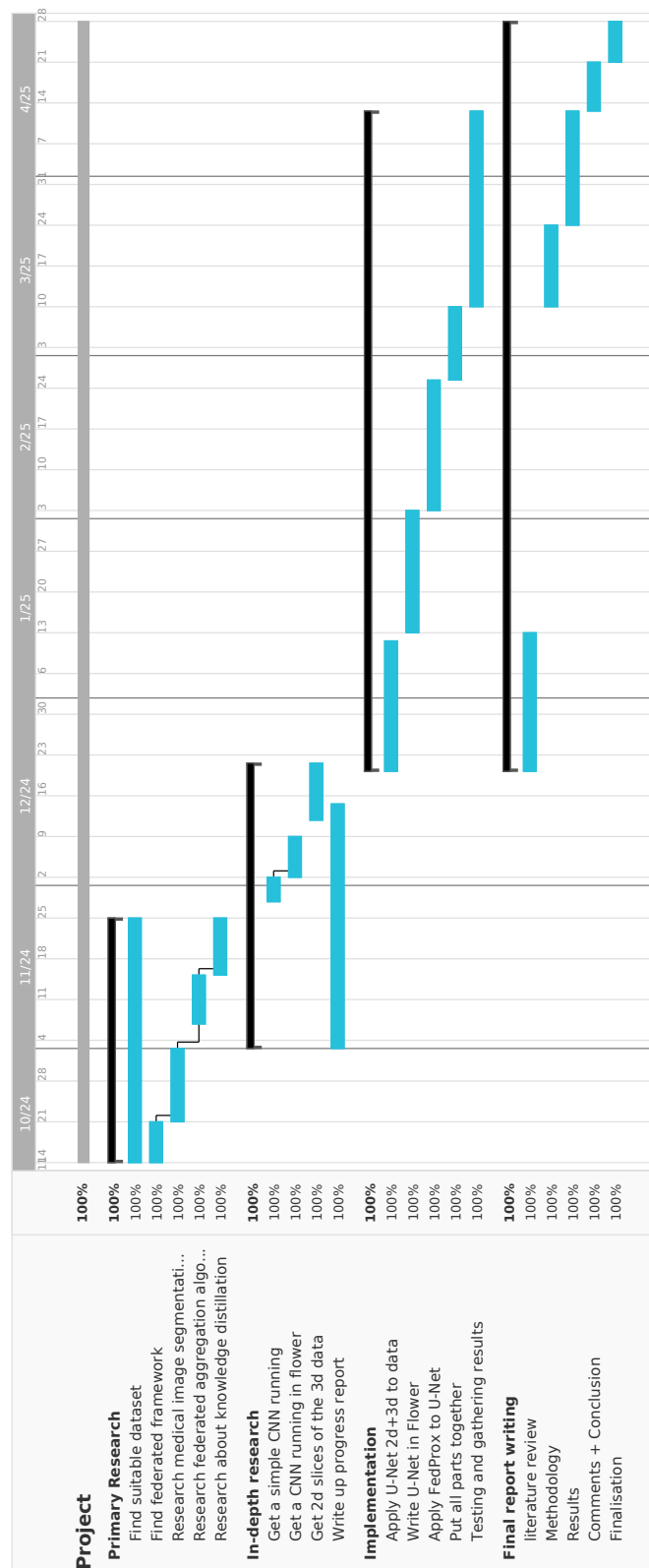
A.1 Table of Contents for Source Code

3D U-Net: main.py, client.py, dataset.py, server.py, model.py, utils.py, **checkpoints**

2D U-Net: main.py, client.py, dataset.py, server.py, model.py, utils.py, **checkpoints**

2D U-Net FedProx: main.py, client.py, dataset.py, server.py, model.py, utils.py, **checkpoints**

A.2 Gantt chart



References

- [1] A. Lerner, K. Palmer, T. Champion, T. O. Millner, E. Scott, C. Lorimer, D. Paraskevopoulos, G. McKenna, S. Marino, R. Lewis, and N. Plowman, "Gliomas in adults: Guidance on investigations, diagnosis, treatment and surveillance," *Clinical Medicine*, vol. 24, p. 100240, 9 2024.
- [2] S. Bakas, M. Reyes, A. Jakab, S. Bauer, M. Rempfler, and A. Crimi, et al, "Identifying the Best Machine Learning Algorithms for Brain Tumor Segmentation, Progression Assessment, and Overall Survival Prediction in the BRATS Challenge," 11 2018.
- [3] S. H. Torp, O. Solheim, and A. J. Skjulsvik, "The WHO 2021 Classification of Central Nervous System tumours: a practical update on what neurosurgeons need to know—a minireview," *Acta Neurochirurgica*, vol. 164, pp. 2453–2464, 9 2022.
- [4] B. H. Menze, A. Jakab, S. Bauer, J. Kalpathy-Cramer, K. Farahani, and J. Kirby, et al, "The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS)," *IEEE Transactions on Medical Imaging* 34(10), vol. 34, pp. 1993–2024, 10 2015.
- [5] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 5 2015.
- [6] Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation," 6 2016.
- [7] F. Isensee, J. Petersen, A. Klein, D. Zimmerer, P. F. Jaeger, S. Kohl, J. Wasserthal, G. Koehler, T. Norajitra, S. Wirkert, and K. H. Maier-Hein, "nnU-Net: Self-adapting Framework for U-Net-Based Medical Image Segmentation," 9 2018.
- [8] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, 1 2019.
- [9] H. Guan, P.-T. Yap, A. Bozoki, and M. Liu, "Federated Learning for Medical Image Analysis: A Survey," 6 2023.

- [10] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," 2 2016.
- [11] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowledge-Based Systems*, vol. 216, 3 2021.
- [12] A. Karargyris, R. Umeton, M. J. Sheller, A. Aristizabal, J. George, A. Wuest, and S. Pati, et al, "Federated benchmarking of medical artificial intelligence with MedPerf," *Nature Machine Intelligence*, vol. 5, pp. 799–810, 7 2023.
- [13] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "FEDERATED OPTIMIZATION IN HETEROGENEOUS NETWORKS," tech. rep., 2020.
- [14] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A Friendly Federated Learning Research Framework," 7 2020.
- [15] M. C. de Verdier, R. Saluja, L. Gagnon, D. LaBella, U. Baid, N. H. Tahon, and M. Foltyn-Dumitru, et al, "The 2024 Brain Tumor Segmentation (BraTS) Challenge: Glioma Segmentation on Post-treatment MRI," 5 2024.
- [16] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao, "Advances and open problems in federated learning," 2021.
- [17] H. Robbins and S. Monro, "A Stochastic Approximation Method," Tech. Rep. 3, 1951.
- [18] J. Kiefer and J. Wolfowitz, "Stochastic Estimation of the Maximum of a Regression Function," Tech. Rep. 3, 1952.
- [19] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, pp. 62–66, 1 1979.
- [20] S. Lakare, "3D Segmentation Techniques for Medical Volumes," tech. rep., 2000.
- [21] D. C. Ciresan, A. Giusti, L. M. Gambardella, and J. Schmidhuber, "Deep Neural Networks Segment Neuronal Membranes in Electron Microscopy Images," in *Advances in Neural Information Processing Systems*, pp. 2852–2860, 2012.

- [22] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," 11 2014.
- [23] S. Bakas, H. Akbari, A. Sotiras, M. Bilello, M. Rozycki, and J. S. Kirby, et al, "Advancing The Cancer Genome Atlas glioma MRI collections with expert segmentation labels and radiomic features," *Scientific Data*, vol. 4, 9 2017.
- [24] P. Team, "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," ACN, 2024.
- [25] NVIDIA, P. Vingelmann, and F. H. Fitzek, "Cuda, release: 10.2.89," 2020.
- [26] F. Wang, R. Jiang, L. Zheng, C. Meng, and B. Biswal, *3D U-Net Based Brain Tumor Segmentation and Survival Days Prediction*, p. 131–141. Springer International Publishing, 2020.
- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.
- [28] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019.
- [29] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farhad, S. Jin, T. Q. S. Quek, and H. V. Poor, "Federated learning with differential privacy: Algorithms and performance analysis," 2019.
- [30] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the ACM Conference on Computer and Communications Security*, pp. 1175–1191, Association for Computing Machinery, 10 2017.
- [31] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning* (H. D. III and A. Singh, eds.), vol. 119 of *Proceedings of Machine Learning Research*, pp. 5132–5143, PMLR, 13–18 Jul 2020.